

МЕЧТАЮТ ЛИ АВТОБОТЫ ОБ АВТОТЕСТАХ?

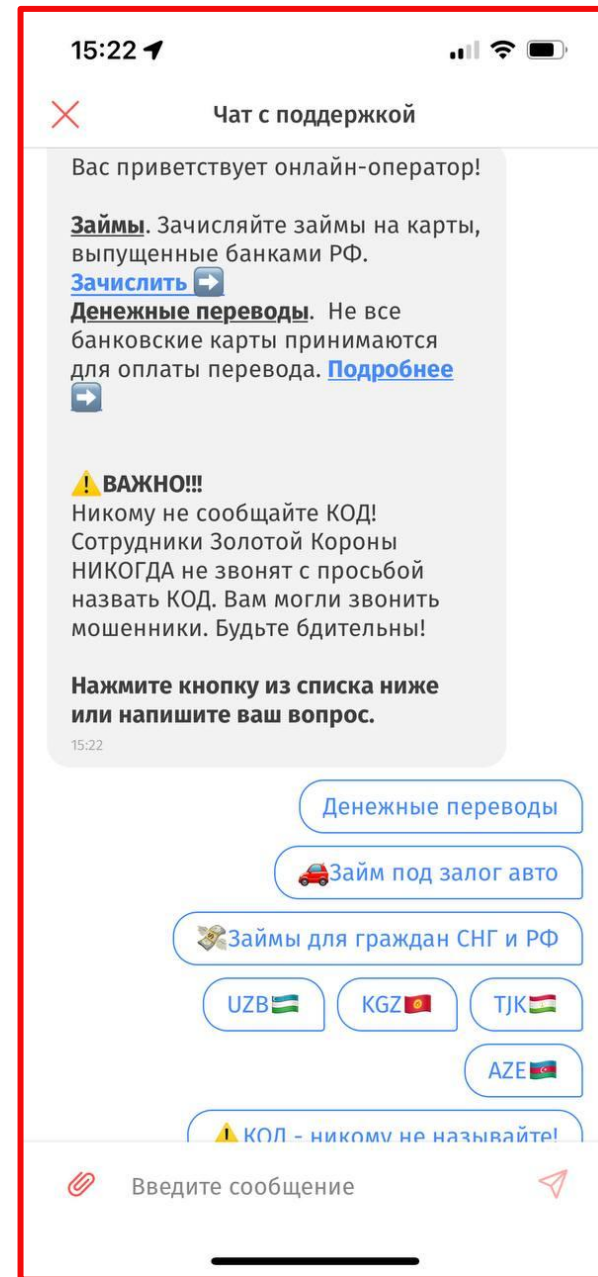
Опыт выстраивания автотестов с нуля с привлечением к процессам manual QA

Игорь Сахно
Ведущий инженер
по тестированию
отдела RnD

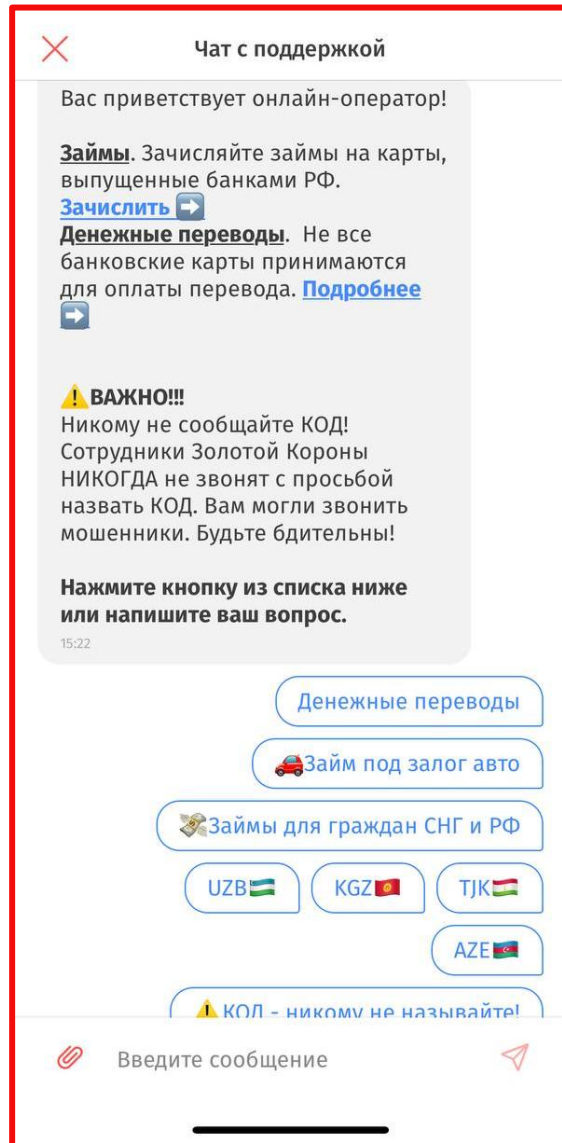


ЧатБот ДП МП «Korona»

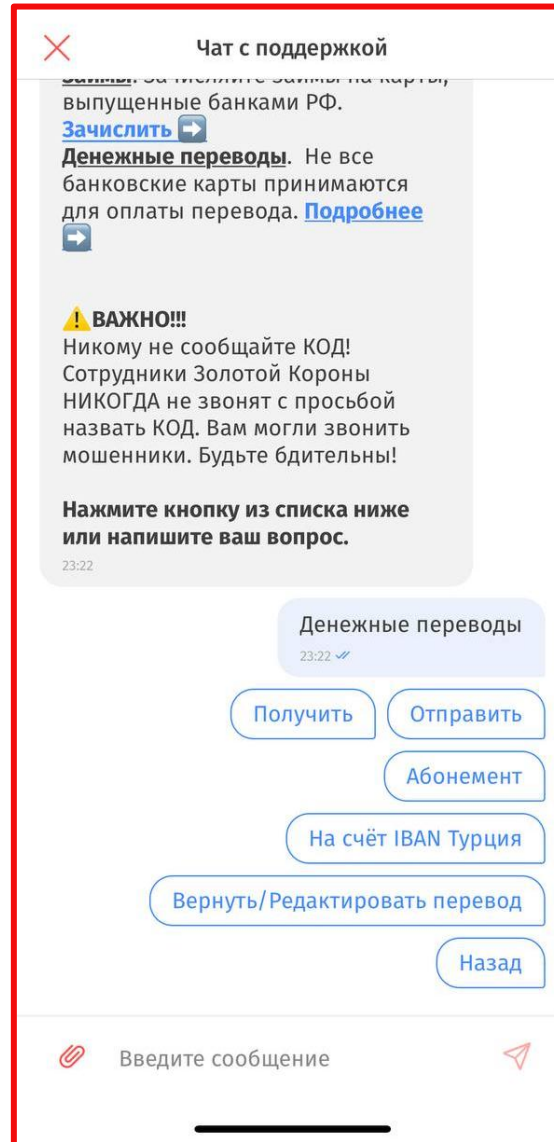
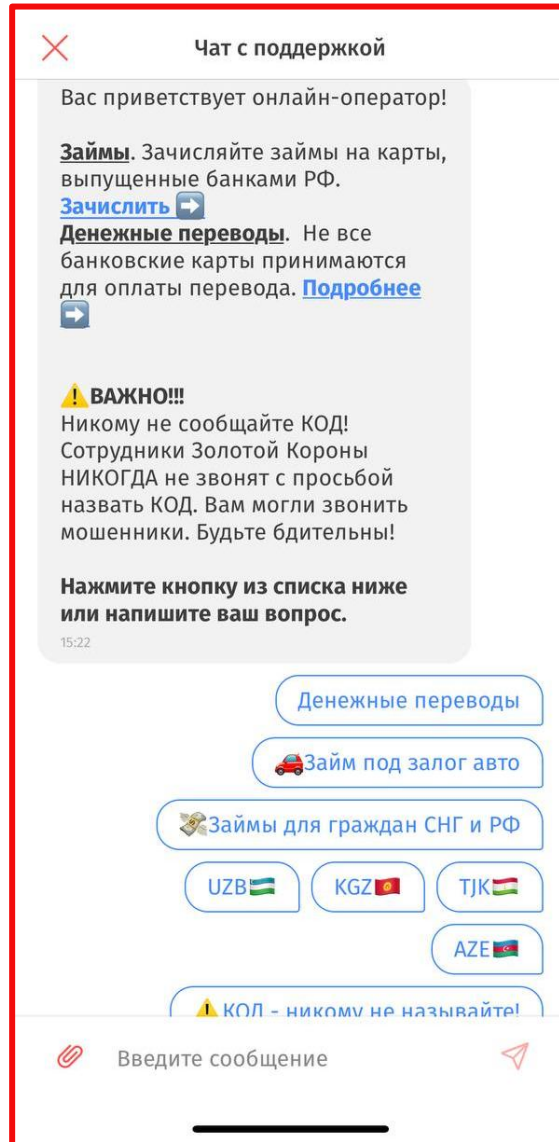
ЧатБот ДП – первая линия обороны между клиентами и операторами техподдержки, цель чатбота — решить возникающие у клиента вопросы, касающиеся денежных переводов и займов, работает на пяти языках, языки и намерения клиентов определяет автоматически



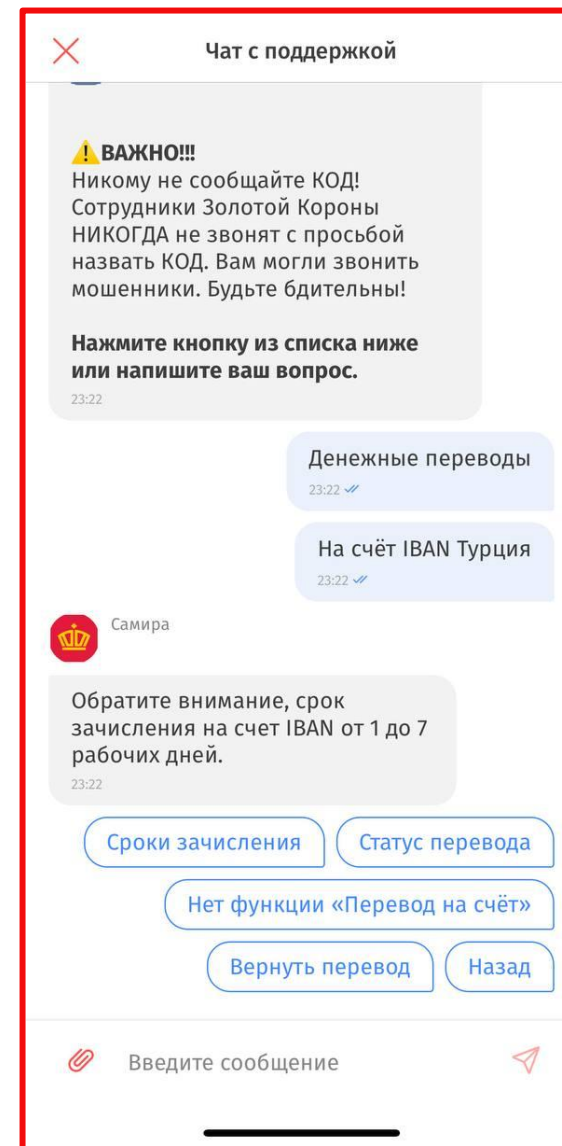
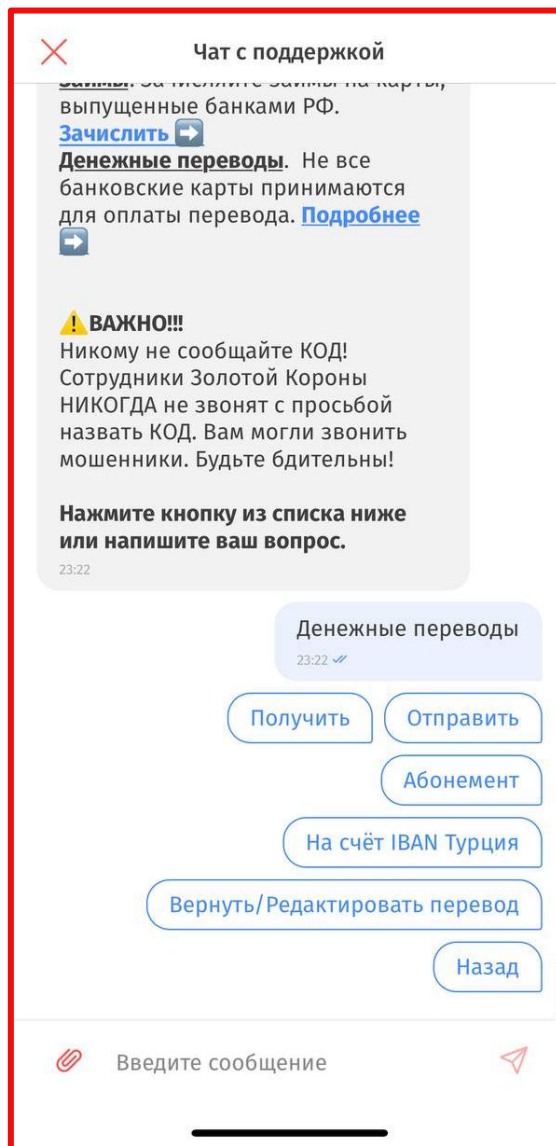
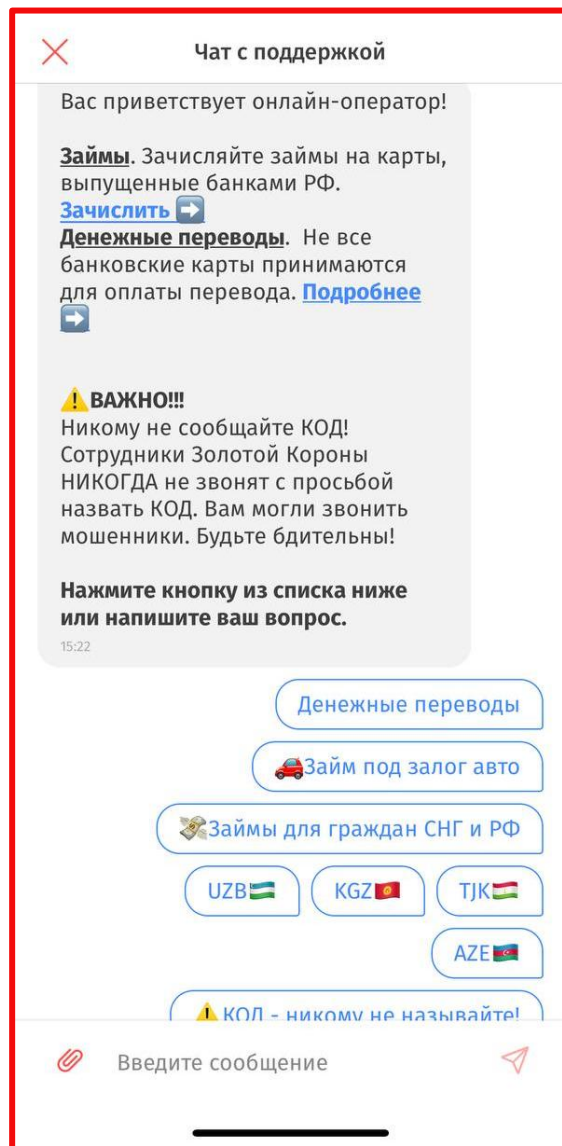
Кнопочное меню



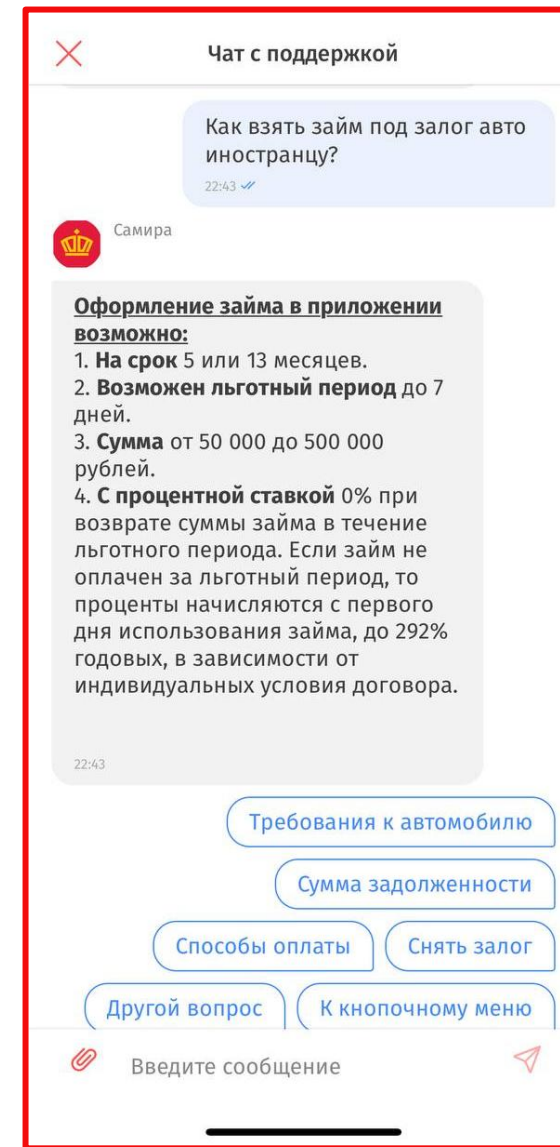
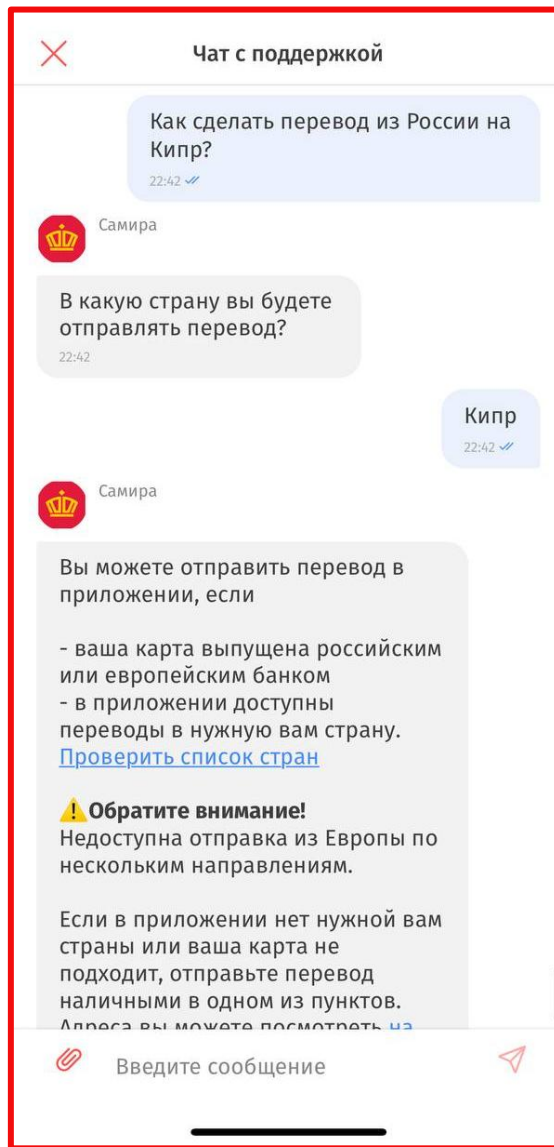
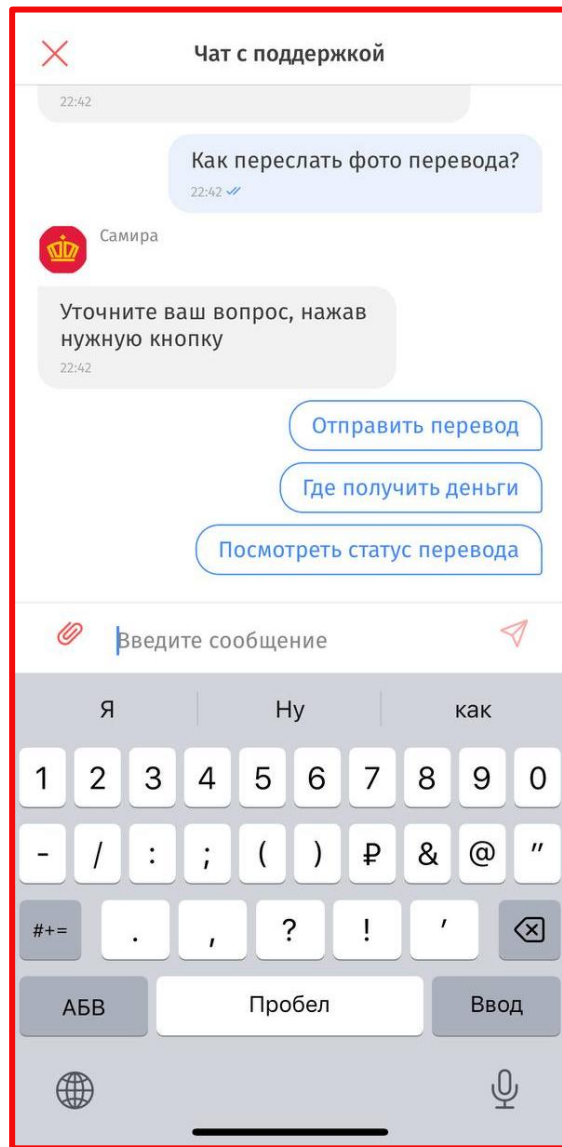
Кнопочное меню



Кнопочное меню

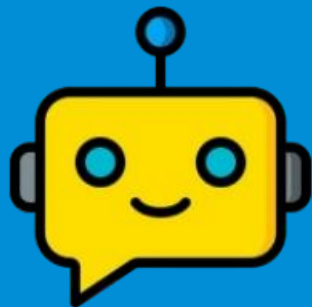


Распознавание текста



korona.tech

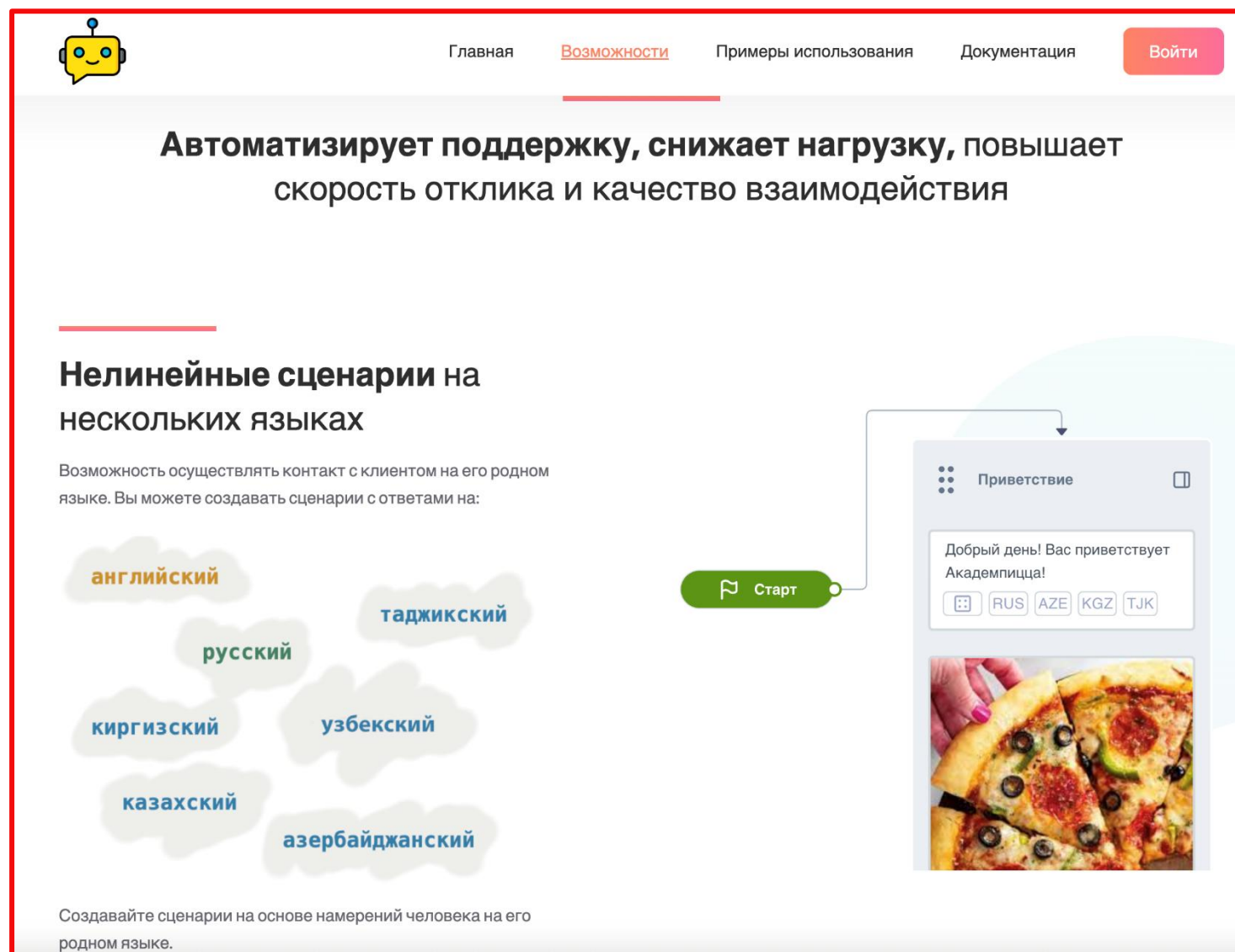
АРМ «АВТОБОТ»



tech



Интерфейс АРМ «Автобот»

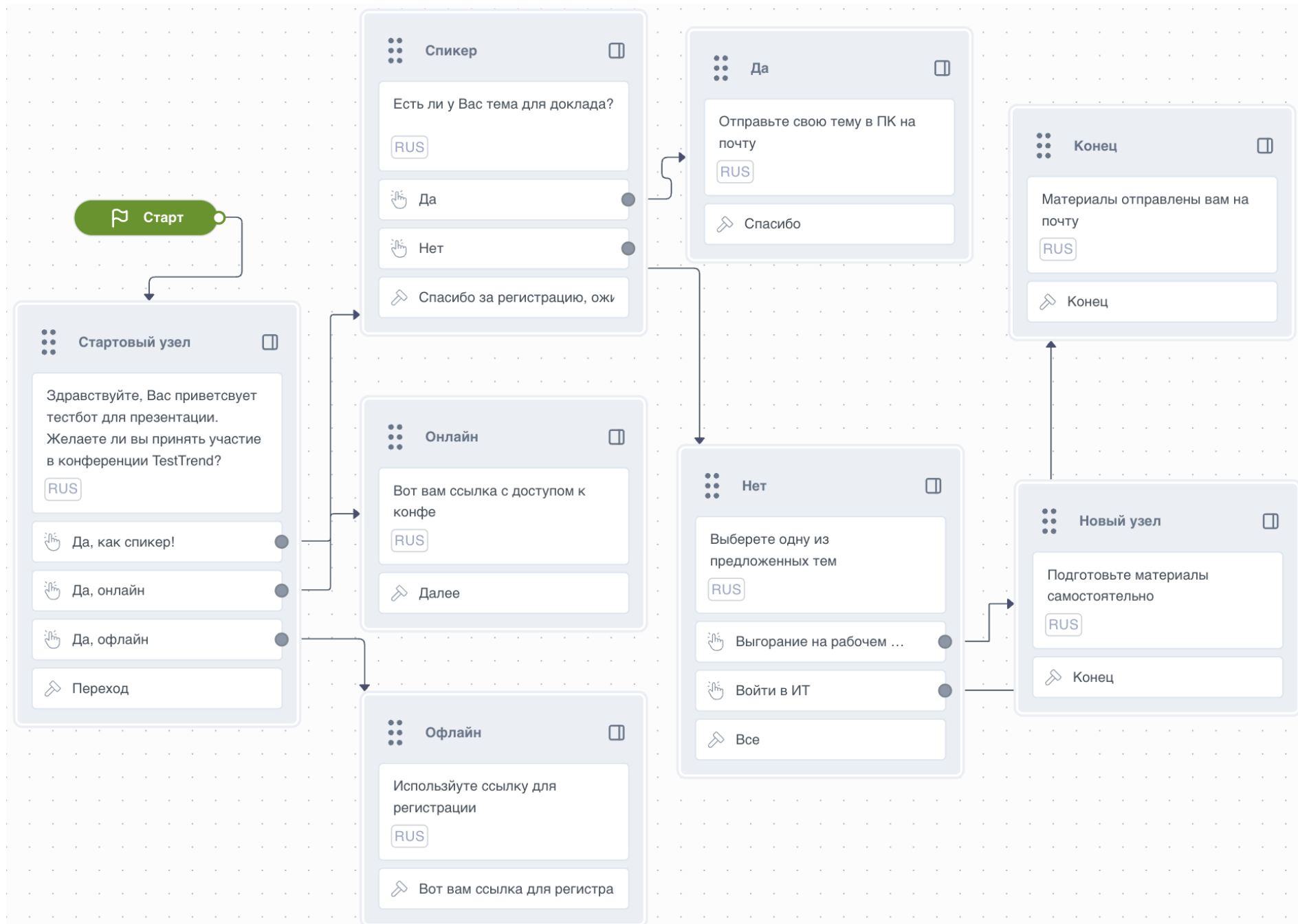


Страница списков сценариев

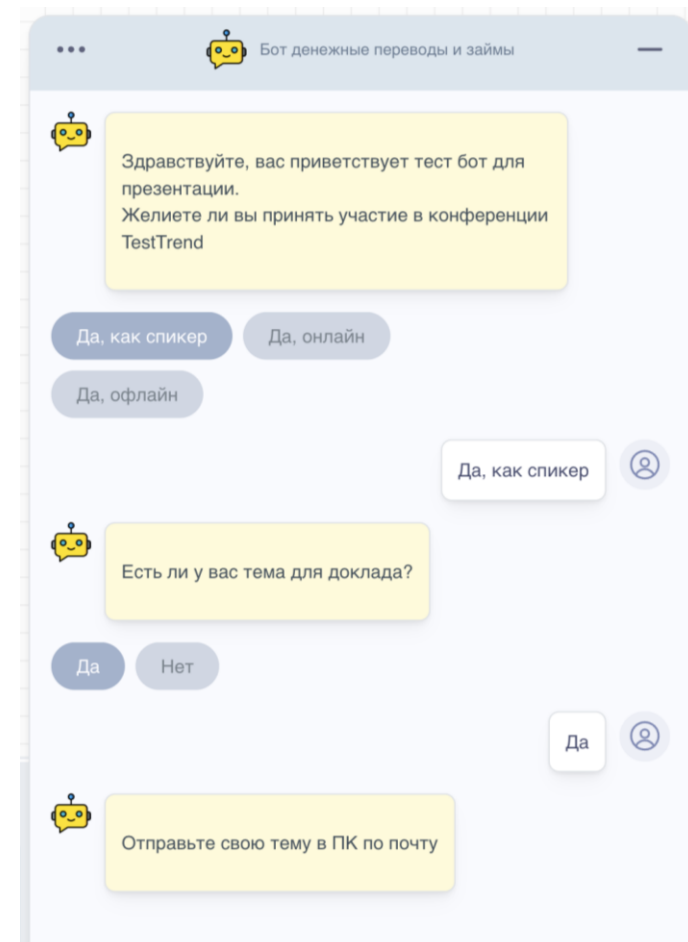
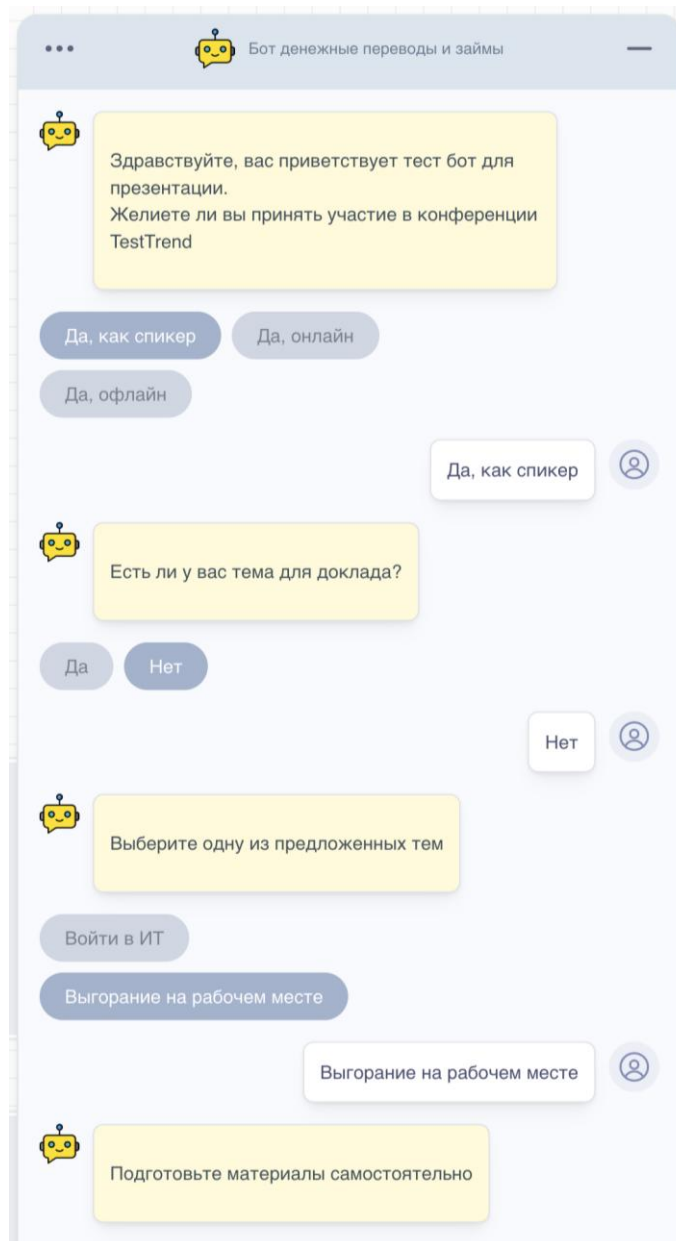
The screenshot shows the Telegram Bot API interface. On the left is a sidebar with navigation options: Главная (Home), Боты (Bots), Каналы (Channels), Сценарии (Scenarios), Мои боты (My bots), Слоты (Slots), Инструменты (Tools), and Посетители (Visitors). The 'Сценарии' option is selected and highlighted. The main area displays a list of scenarios under the heading 'Сценарии'. A table with two columns, 'Название' (Name) and 'Интененты' (Intents), lists the scenarios. The 'Test Demo' scenario is highlighted with a red box. Below the table is a section for 'Сценарии' with a plus icon and a list of scenarios.

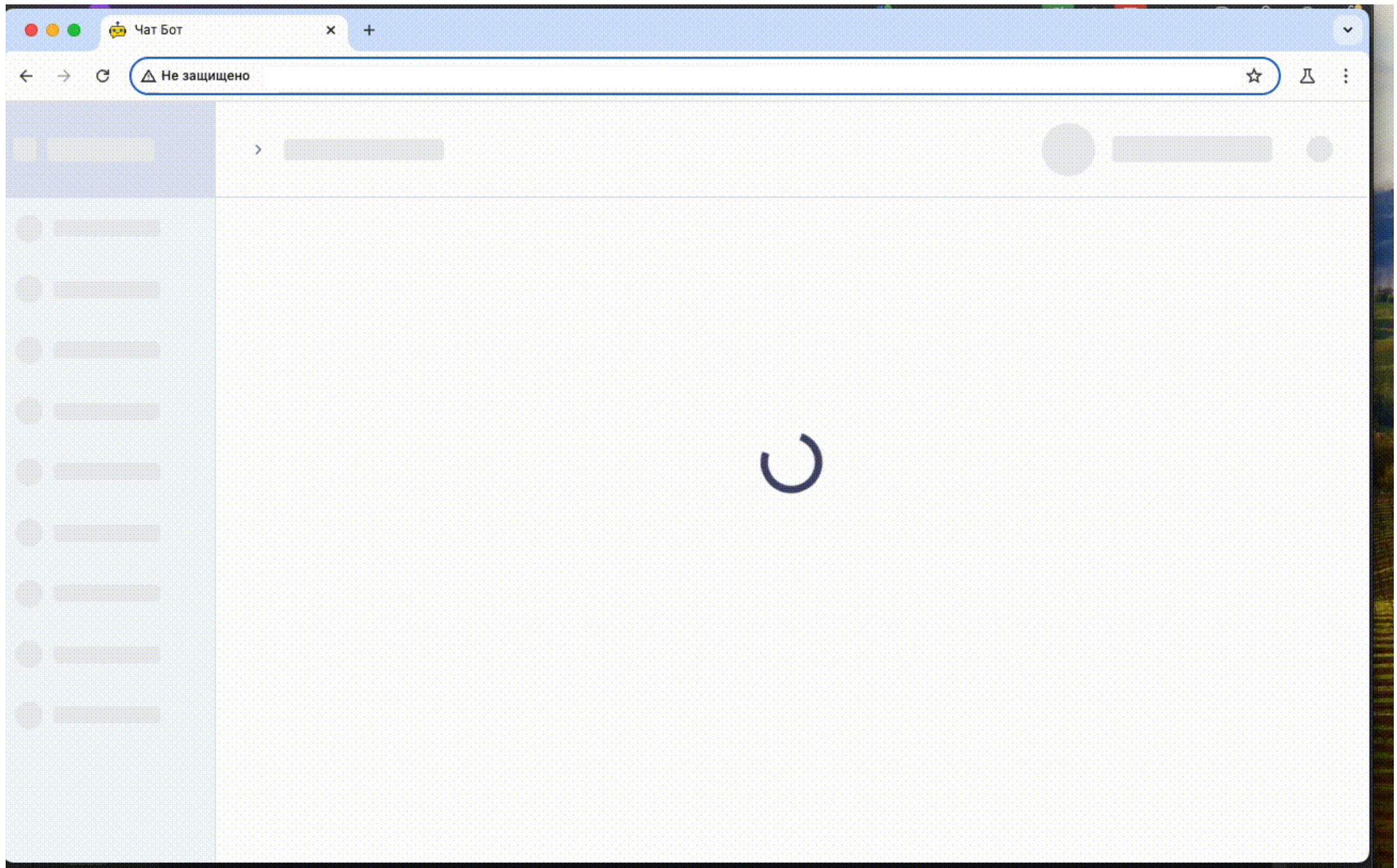
Название	Интененты
Test Demo	—
code_error_test	—
korona_card2_buttons	—
abonement_info	—
close_dialog_kpe	—
dp_buttons_v2	—

Страница создания дерева сценариев



Режим тестирования сценариев





korona.tech

Теперь давайте поговорим о том,
как нам обеспечить качество
для такого проекта

tech



Как тестировать АРМ «Автобот»?

- На старте проекта был только бекенд, проводили ручное тестирование
- Разработчики покрыли большую часть бекенда API тестами (Unit, Integration)
- При старте разработки фронтенда – ручное тестирование
- Т.к. планы на развитие продукта большие, появилась необходимость разработки автотестов, необходимо было выбрать стек технологий
- Не было готовых решений

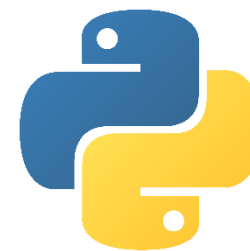
Проблематика – мотивация

- Проект развивается и усложняется, тем самым увеличивается объем регрессионного тестирования перед релизами, что увеличивает время на выкатку релизов
- Т.к. все успешно протестированные ветки мержим в мастер, появилась необходимость автоматически отслеживать качество на мастер-ветке
- Ускорить выкатку релизов при обеспечении должного качества
- Повышение квалификации Manual QA в автоматизации

Проблемы, которые необходимо решить на старте автотестов

1. Как писать автотесты на APM «Автобот»? – ресерчу стек технологий
2. Как и когда запускать автотесты?
3. Кто и где сможет запускать их?
4. Как доносить информацию до команды, как реализовать отчетность?
5. Как внедрить автотесты в рабочий процесс?

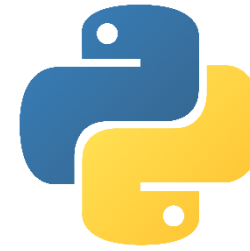
Стек технологий



1. RnD – разработка на Python, есть небольшой опыт

Стек технологий для тестов

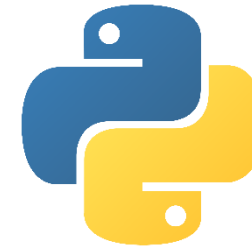
1. RnD – разработка на Python, есть небольшой опыт
2. Фреймворк для взаимодействия с UI – Playwright



Playwright

Стек технологий для запуска

1. RnD – разработка на Python, есть небольшой опыт
2. Фреймворк для взаимодействия с UI – Playwright
3. Запуск – GitLab

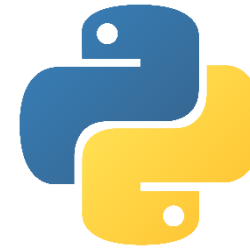


Playwright



Стек технологий для отчетности

1. RnD – разработка на Python, есть небольшой опыт
2. Фреймворк для взаимодействия с UI – Playwright
3. Запуск – GitLab
4. Отчетность – Allure
5. Предоставление отчетности – eXpress



Playwright



korona.tech

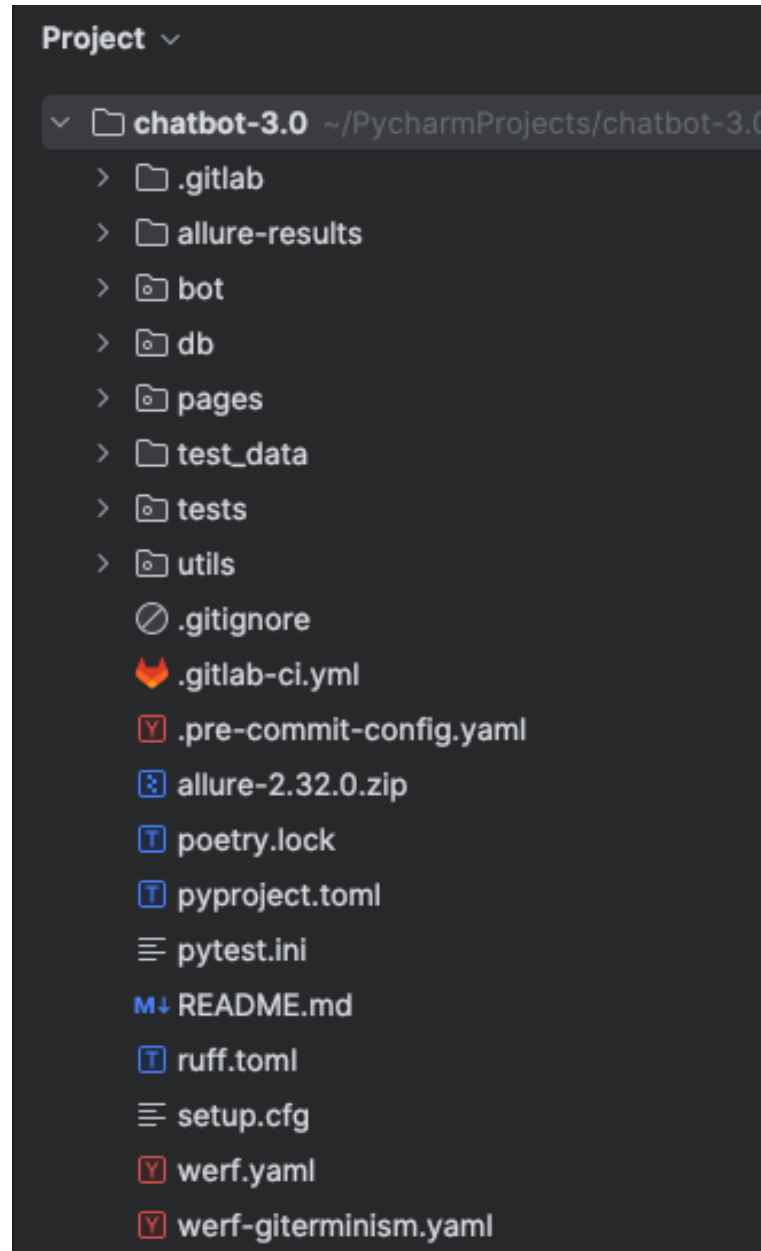
PAGEOBJECTMODEL



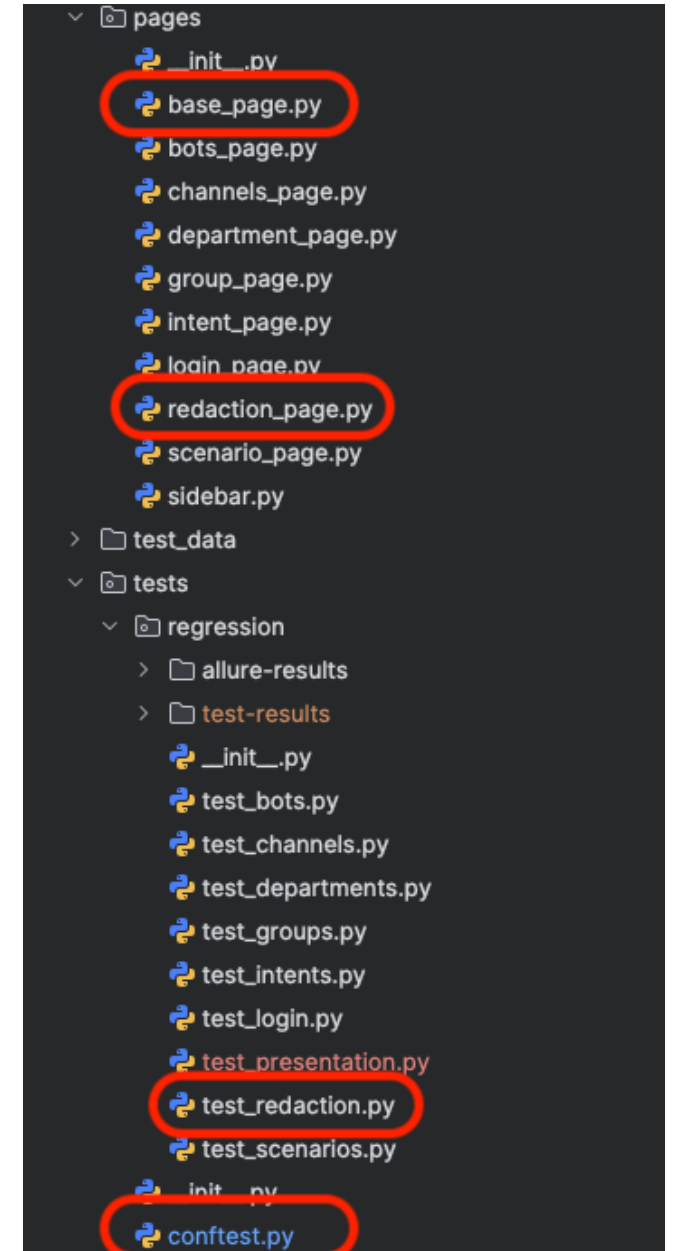
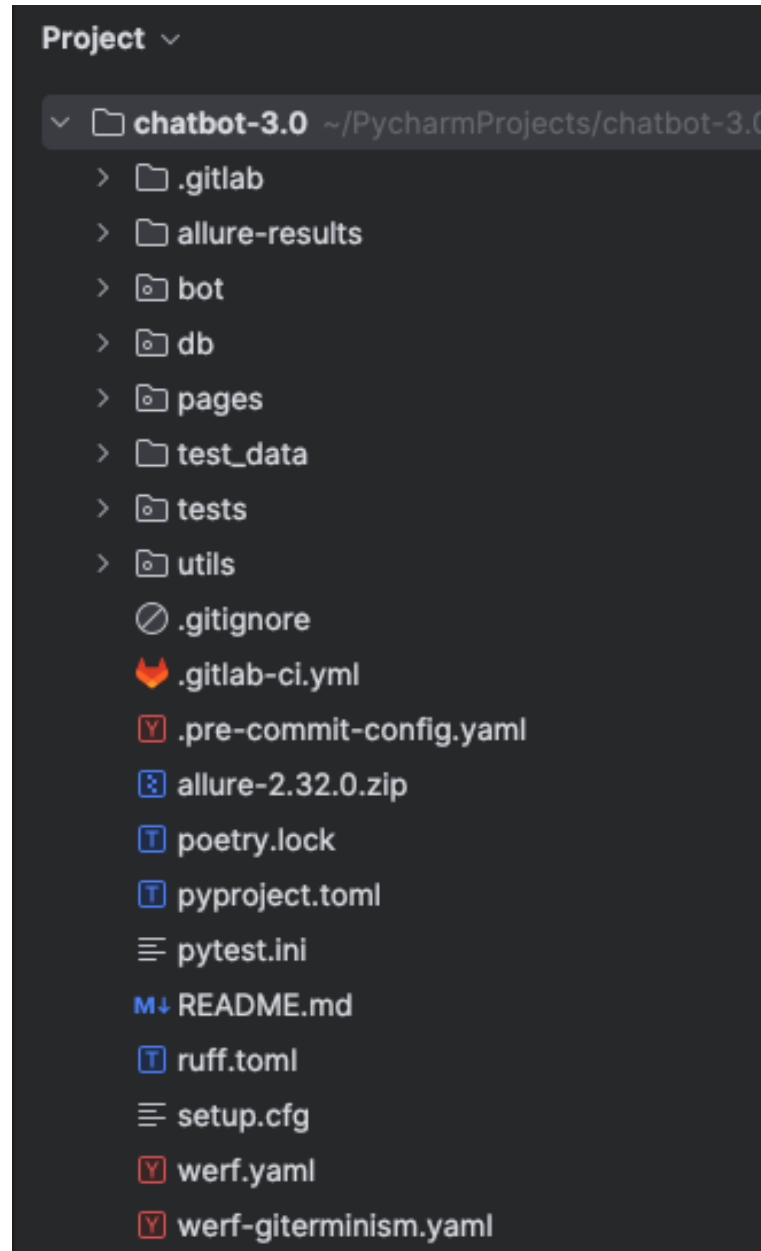
tech



POM – PageObjectModel



POM – PageObjectModel



POM – PageObjectModel

```
class ListRedactionsPage(BasePage): 2 usages  @ i.sakhno
    """Содержит локаторы и методы для страницы Редакции Сценариев"""

    def __init__(self, page: Page): @ i.sakhno
        super().__init__(page)
        self.btn_OPEN_SIDE_BAR = page.get_by_test_id('table-button-open-side-bar')
        self.col_UUID = page.get_by_test_id('column-name-redaction-uuid')
        self.fld_STATE_COLOR = page.get_by_test_id('table-field-redaction-status-color')
        self.fld_UUID = page.get_by_test_id('table-field-redaction-uuid')
        self.btn_SIDE_BAR_EDIT = page.get_by_role(role: 'button', name='Редактировать')
        self.btn_SIDE_BAR_ACTIVATE = page.get_by_role(role: 'button', name='Активировать')

    @allure.step('Получить количество версий') 2 usages (2 dynamic)  @ i.sakhno
    def get_versions_count(self) -> int:
        elements = self.fld_UUID.all()
        count = len(elements)
        return count
```

POM – PageObjectModel

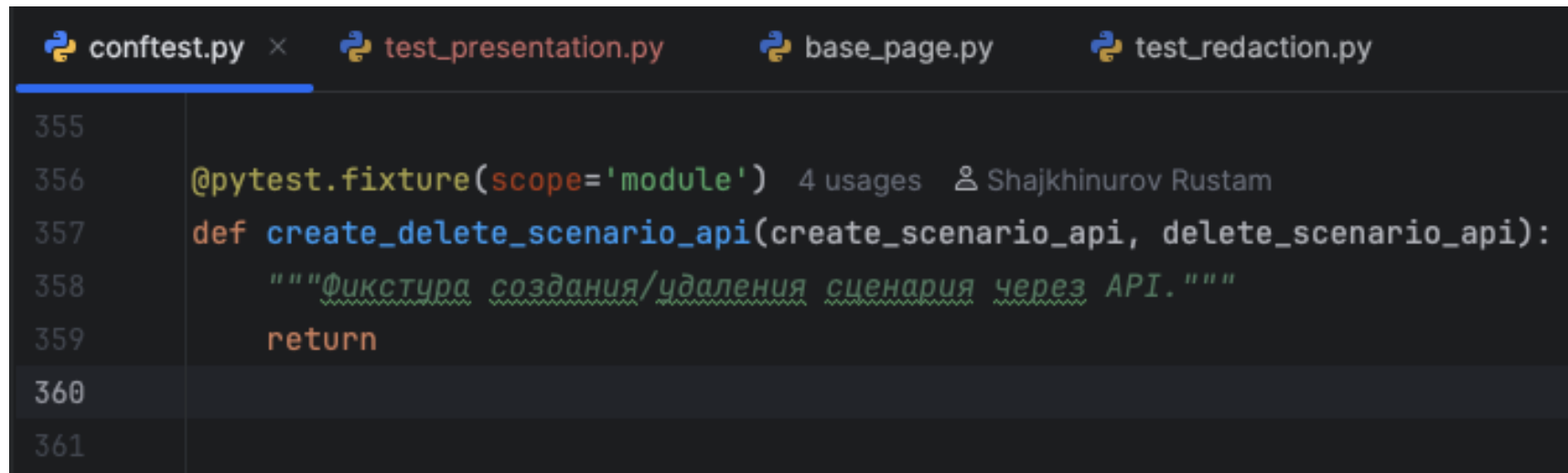
```
@allure.title('Регрессионный тест редакции сценария')  @ i.sakhno +5 *
@pytest.mark.usefixtures('select_actual_namespace')
@pytest.mark.usefixtures('ui_login_logout')
class TestScenarioRedactions:

    @pytest.mark.regression  @ i.sakhno +4
    @pytest.mark.redaction
    @allure.feature('Сценарии')
    @allure.story('Регресс редакции сценария')
    @pytest.mark.usefixtures('create_delete_scenario_api')
    @pytest.mark.usefixtures('create_delete_channel_api')
    @pytest.mark.usefixtures('create_bot_api')
    @pytest.mark.usefixtures('create_delete_department_api')
    @pytest.mark.usefixtures('get_model_classification_api')
    @pytest.mark.usefixtures('create_group_api')
    def test_redaction(self, side_bar, temp_data, scenarios_page, redactions_page, canvas, config, tag_factory):
        """Проверка создания нод и перетаскивания элементов на канвасе."""
        with allure.step('Создание тегов необходимых для теста'):
            for name in ['base_node', 'button', 'condition']:
                tag_factory(name=name)

        with allure.step('Открытие страницы списка сценариев'):
            side_bar.navigate_to_scenarios()

        with allure.step('Проверка: группа выбрана в неймспейсе'):
            expect(side_bar.fld_NAMESPACE, 'Группа не выбрана').to_have_text(temp_data['group_name'])
```

PyTest-фикстуры. Подготовка данных для теста



The screenshot shows a code editor with four tabs: `conftest.py`, `test_presentation.py`, `base_page.py`, and `test_redaction.py`. The `conftest.py` tab is active. The code in the editor is as follows:

```
355
356 @pytest.fixture(scope='module') 4 usages Shajkhinurov Rustam
357 def create_delete_scenario_api(create_scenario_api, delete_scenario_api):
358     """Фикстура создания/удаления сценария через API."""
359     return
360
361
```

PyTest-фикстуры. Подготовка данных для теста

```
conftest.py x test_presentation.py base_page.py test_redaction.py
> Q- 0 results ↑ ↓ 🔍 ⋮
743
744 @pytest.fixture(scope='module') 1 usage Shajkhinurov Rustam *
745 def create_scenario_api(temp_data, headers_3_0, scenario_payload, config) -> requests.Response:
746     """Фикстура создания сценария через API."""
747     response = requests.post(
748         url=f'{config.backend_host}/scenarios',
749         headers=headers_3_0,
750         json=scenario_payload,
751         timeout=10,
752     )
753     temp_data['scenario_name'] = response.json()['name']
754     temp_data['scenario_code'] = response.json()['scenario_name']
755     temp_data['scenario_id'] = response.json()['id']
756     return response
757
758
759 @pytest.fixture(scope='module') 1 usage Shajkhinurov Rustam +1
760 def delete_scenario_api(get_scenario_api, temp_data, headers_3_0, config):
761     """Фикстура удаления сценария через API."""
762     yield
763     requests.delete(url=f'{config.backend_host}/scenarios/{temp_data["scenario_id"]}', headers=headers_3_0, timeout=10)
764
```

korona·tech

ALLURE REPORTS



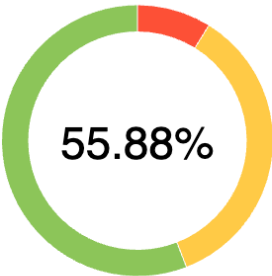
tech



Allure Reports

ALLURE REPORT 15.12.2025
2:37:51 - 2:48:17 (10m 25s)

34
тестовых сценария



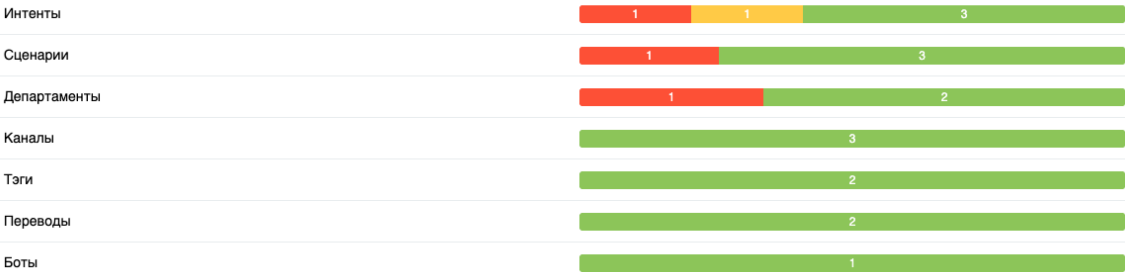
ТЕСТ СЮИТЫ Всего 1 элемент



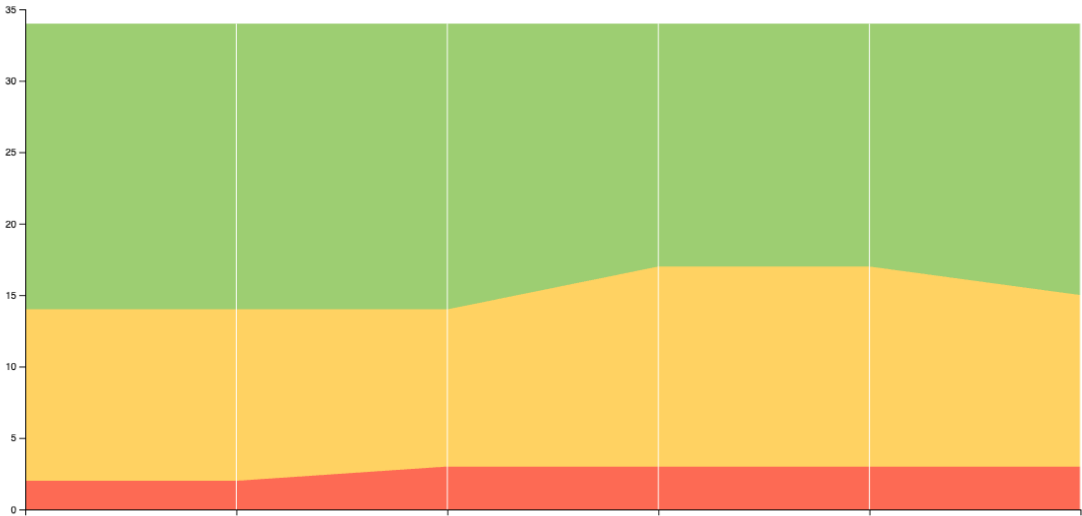
ОКРУЖЕНИЕ

python_version	Python 3.13.5
master_page	

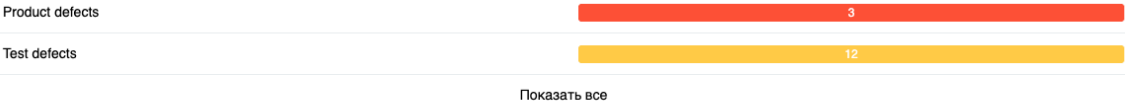
ФУНКЦИОНАЛЬНОСТЬ Всего 9 элементов



ТРЕНД



КАТЕГОРИИ Всего 2 элемента



СИСТЕМА ВЫПОЛНЕНИЯ ТЕСТОВ



chatbot-3.0

Allure Reports

Behaviors

order	name	duration	status	Status: 0 1 11 0 0	Marks:     
>	Группы			2	
>	Департаменты			1	
▼	Интененты			1 2	
>	Регресс интенентов			1	
>	Регресс отображение канала в списке интенентов			1	
▼	Регресс фильтр интенентов			1	
	#1 test_filter_intent_by_model[chromium] 'chromium'			30s 327ms	
>	Каналы			2	
>	Логин			1	
>	Сценарии			3	

tests.regression.test_intents.TestIntents#test_filter_intent_by_model

Broken  test_filter_intent_by_model[chromium]

Overview History Retries

playwright._impl._errors.TimeoutError: Locator.click: Timeout 30000ms exceeded.
Call log:
- waiting for get_by_role("option", name="yzdul8p_test_group_20250724072420")

Tags: [intents](#) [regression](#) [filters](#)

Categories: Test defects

Severity: normal

Duration: 30s 327ms

Description

Проверка фильтров на странице интенентов.

Parameters

browser_name: 'chromium'

Execution

> Set up

▼ Test body

- ✓ Открыть страницу списка интенентов 111ms
- ✓ Проверка: Страница списка интенентов открыта 170ms
- ▼ Выбор классификатора в выпадающем списке 2 sub-steps 30s 046ms
 - ✓ Кликнуть по фильтру Модель 43ms
 - ▼ Выбор элемента из выпадающего списка 30s 002ms

playwright._impl._errors.TimeoutError: Locator.click: Timeout 30000ms exceeded.
Call log:
- waiting for get_by_role("option", name="yzdul8p_test_group_20250724072420")

playwright._impl._errors.TimeoutError: Locator.click: Timeout 30000ms exceeded.
Call log:
- waiting for get_by_role("option", name="yzdul8p_test_group_20250724072420")

Allure Step

```
start_point = canvas.nod_START_POINT.bounding_box()
with allure.step('Создание первой ноды перетаскиванием из стартовой'):
    canvas.drag_and_drop(canvas.nod_START_POINT, start_point, 150, 150)
```

Allure Step

```
start_point = canvas.nod_START_POINT.bounding_box()
with allure.step('Создание первой ноды перетаскиванием из стартовой'):
    canvas.drag_and_drop(canvas.nod_START_POINT, start_point, 150, 150)
```

```
test.py  test_presentation.py  base_page.py  test_redaction.py  redaction_page.py ×
class Canvas(BasePage): 2 usages i.sakhno

    @allure.step('Перетаскивание ноды по канвасу по координатам') 3 usages (3 dynamic) i.sakhno
    def drag_and_drop(self, locator, box, x, y):
        locator.hover()
        self.page.mouse.down()
        self.page.mouse.move(box['x'] + x, box['y'] - y) # смещение от начальной точки
        self.page.mouse.up()
```

Allure Step

```
start_point = canvas.nod_START_POINT.bounding_box()
with allure.step('Создание первой ноды перетаскиванием из стартовой'):
    canvas.drag_and_drop(canvas.nod_START_POINT, start_point, 150, 150)
```

✓ Создание первой ноды перетаскиванием из стартовой 1 вложенный шаг

✓ Перетаскивание ноды по канвасу по координатам 4 параметра

locator	<Locator frame=<Frame name= url='http://rnd-chatbot-frontend-master.mo-test.dp-k8s-test.ftc.ru:30101/app/sc
box	{'x': 892.8539428710938, 'y': 427.21295166015625, 'width': 11.5740966796875, 'height': 11.5740966796875}
x	150
y	150

```
@allure.step('Перетаскивание ноды по канвасу по координатам')
def drag_and_drop(self, locator, box, x, y):
    locator.hover()
    self.page.mouse.down()
    self.page.mouse.move(box['x'] + x, box['y'] + y)
    self.page.mouse.up()
```

korona.tech

ЛОКАТОРЫ DATA-TESTID



Playwright

tech



Локаторы data-testid

```
✓ class Canvas(BasePage): 2 usages i.sakhno
    """Содержит локаторы и методы для страницы Канваса и боковых панелей"""

✓ def __init__(self, page): i.sakhno
    super().__init__(page)
    self.nod_START = page.get_by_test_id('canvas-start-node')
    self.img_START_NODE_FLAG = page.get_by_test_id('canvas-start-node-flag')
    self.nod_START_POINT = page.get_by_test_id('canvas-start-node-point')
    self.nod_CARD = page.get_by_test_id('node-card')
    self.btn_CENTER_IMAGE = page.get_by_role(role='button', name='fit view')
    self.btn_ZOOMOUT = page.get_by_title('zoom out')
```

Локаторы data-testid

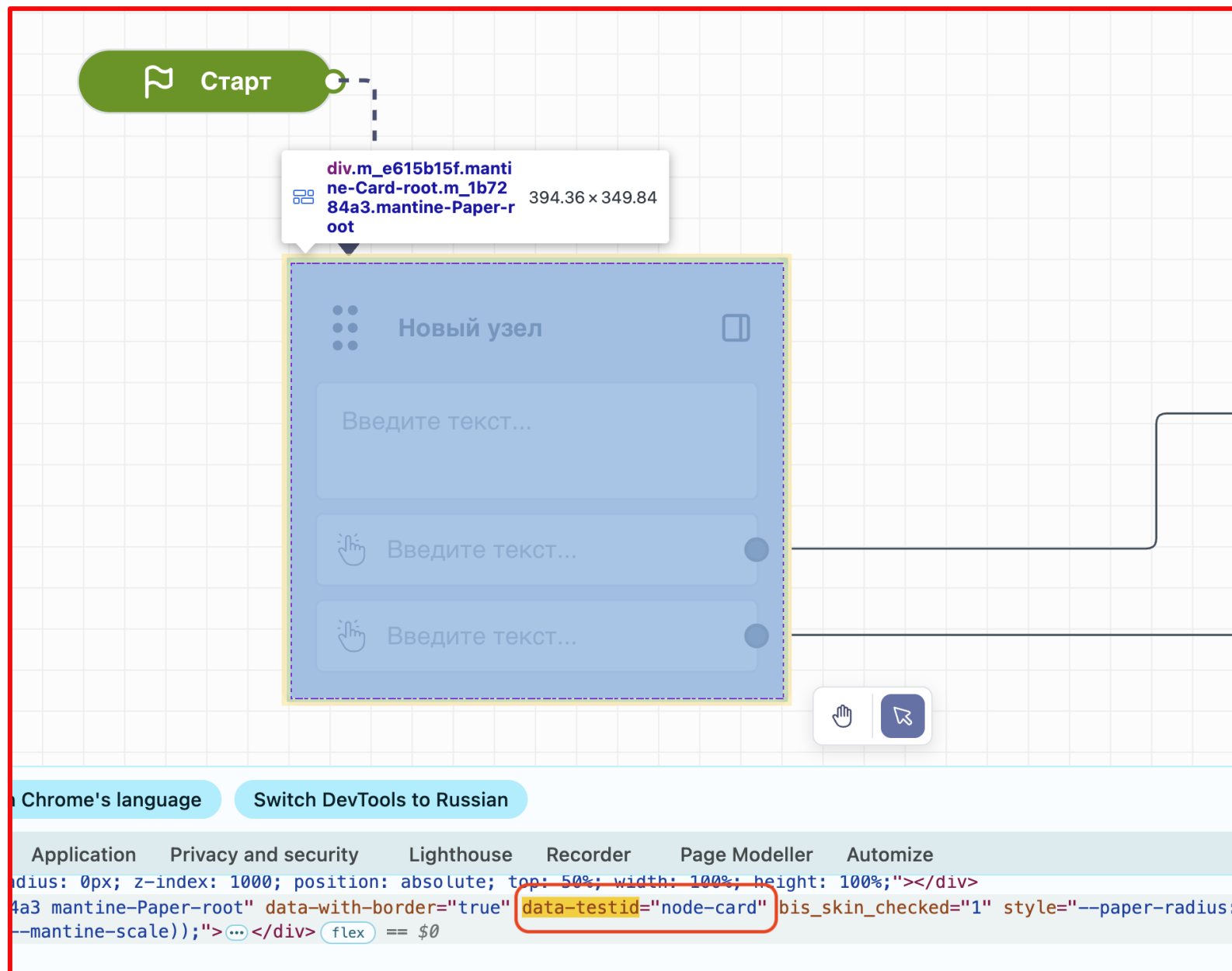
src/utils/constants/testIds.ts

```
125 + NODE_INPUT_BUTTON_FIELD: 'node-input-button-field',
126 + NODE_INPUT_CONDITION_FIELD: 'node-input-condition-field',
127 + NODE_INPUT_BUTTON_POINT: 'node-input-button-point',
128 + NODE_INPUT_CONDITION_POINT: 'node-input-condition-point',
129 + NODE_CARD: 'node-card',
130 + TAB_MAIN_PARAMS: 'tab-main-params',
131 + TAB_MAIN_PARAMS_DESCRIPTION: 'tab-main-params-description',
132 + TAB_ACTION: 'tab-action',
133 + TAB_RESPONSES: 'tab-responses',
```

src/routes/_auth/scenarios/\$scenarioId/editor/\$scenarioVersionId/-components/ScenarioEditor

```
↑
48     }}
49   />
50   <Card
51 +   data-testid={TEST_IDS.NODE_CARD}
52     withBorder
53     p={2}
54     radius='md'
55   />
@@ -70,6 +72,7 @@ export const ScenarioEditorNode = memo((props: ScenarioEditorNodeProps) => {
72   >
73   <Flex align='center' justify='space-between'>
```

Локаторы data-testid



korona.tech

ЗАПУСК ТЕСТОВ GITLAB



tech



Ручной запуск на ветке

```
Run Custom Tests:
  image: "${CI_REGISTRY_IMAGE}:testing-${CI_COMMIT_SHORT_SHA}"
  stage: tests
  allow_failure: true
  rules:
    - if: '$CI_PIPELINE_SOURCE == "web"'      # Ручной запуск
    - if: '$CI_PIPELINE_SOURCE == "push"'     # При push
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"' # При MR
    - when: never # В остальных случаях не запускать
  variables:
    TEST_KEYWORD: "" # Default: empty (run all)
    TEST_MARKER: "" # Default: empty (run all)
    BRANCH: "master" # Default: empty (master)
  script:
    - echo "$TEST_KEYWORD"
    - echo "$TEST_MARKER"
    - echo "$BRANCH"
    - pytest -v -s -k "$TEST_KEYWORD" -m "$TEST_MARKER" --branch="$BRANCH" --alluredir allure-results-manual
  artifacts:
    when: always
    paths: [ 'allure-results-manual/' ]
    expire_in: 1 day
```

Ручной запуск на ветке

Run pipeline

Run for branch name or tag

master ▾

Variables

Variable ▾

TEST_MARKER

login



Маркер тестов, например: regression, login, scenarios

Variable ▾

TEST_KEYWORD

Input variable value



Ключевое слово для фильтрации тестов, например: filter, login, create

Variable ▾

BRANCH

qpml-777



Ветка фронтенда, на которой запускать автотесты, прим. qpml-1234

Variable ▾

Input variable key

Input variable value

Specify variable values to be used in this run. The variables specified in the configuration file as well as [CI/CD settings](#) are used by default.

Variables specified here are **expanded** and not **masked**.

Run pipeline

Cancel

Автоматический ночной запуск на мастере

```
Testing:
  image: "${CI_REGISTRY_IMAGE}:testing-${CI_COMMIT_SHORT_SHA}"
  stage: tests
  allow_failure: true
  script:
    - pytest
  rules:
    - if: '$CI_PIPELINE_SOURCE == "schedule"' # Только по расписанию
      when: always
    - when: never # В остальных случаях не запускать
  artifacts:
    when: always
    paths: [ 'allure-results/' ]
    expire_in: 1 day
```

Автоматический ночной запуск на мастере

1

Search or go to...

Project

chatbot-e2e

Pinned

Manage

Plan

Code

Build

Pipelines

Jobs

Pipeline editor

Pipeline schedules

All 1 Active Inactive

New schedule

Description	Target	Last Pipeline	Next Run	Owner	
Regression night run	master	Passed	in 2 hours		

Автоматический ночной запуск на мастере

Edit Pipeline Schedule

Description

Regression night run

Interval Pattern

- ☐ Every day (at 11:14pm)
- ☐ Every week (Thursday at 11:14pm)
- ☐ Every month (Day 1 at 11:14pm)
- ☒ Custom

30 02 * * *

Set a custom interval with Cron syntax. [What is Cron syntax?](#)

Cron timezone

[UTC+3] Moscow

Select target branch or tag

master

Variables

Variable

Input variable key

Input variable value

☒ Activated

Save changes

Cancel

korona.tech

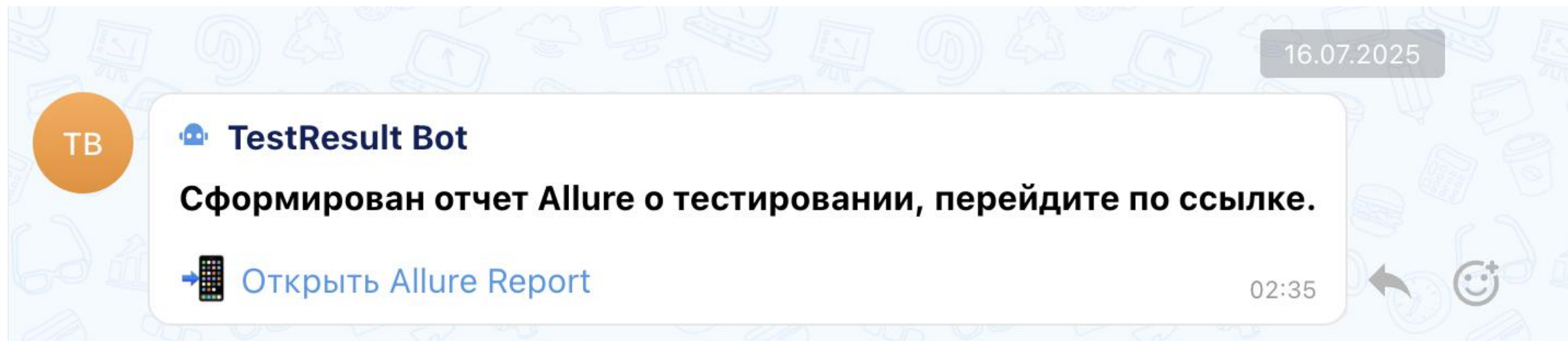
ОТЧЕТЫ В EXPRESS

eXpress

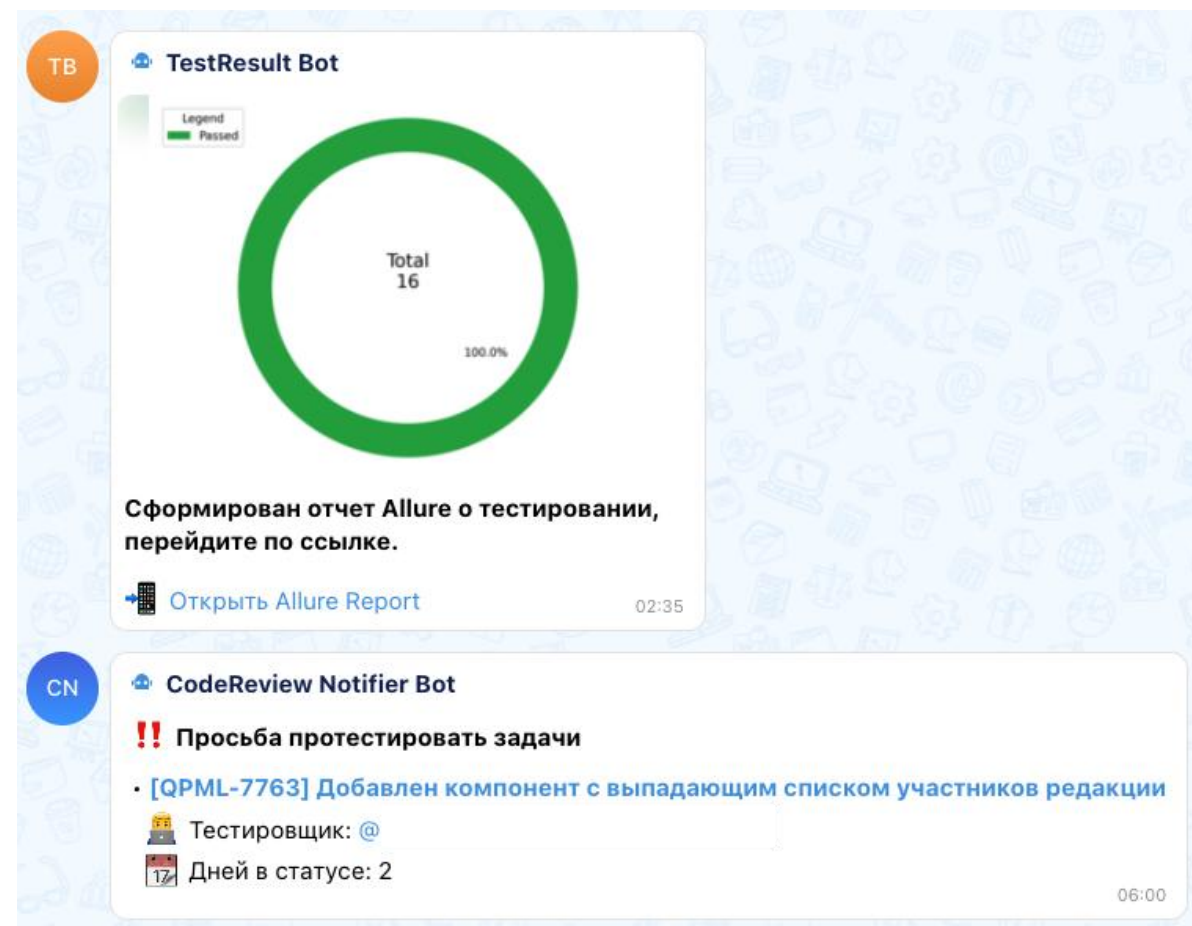
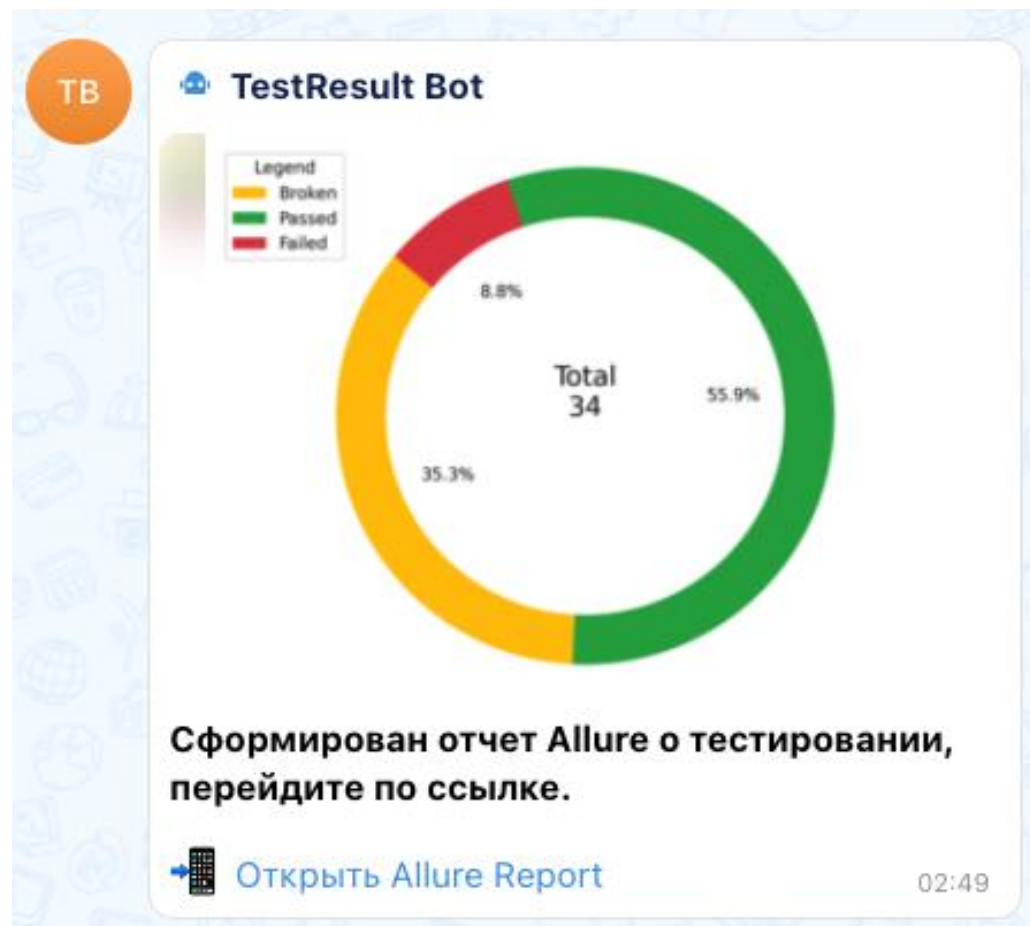
tech



Отчеты в eXpress



Отчеты в eXpress



Выводы

- Однозначно, автоботы не только мечтают, но и сильно нуждаются в автотестах
- Нетиповой проект, в котором используется как UI, так и API, на который получилось написать фреймворк и подключить команду, передать знания Manual QA
- Я показал на своем примере и реализованном проекте, что не так уж и сложно было начать автоматизацию в команде без большого опыта работы с автотестами. Определил состав работ на бекенд – подготовка данных, на фронтенд – добавление локаторов, сами тесты, запуск и отчетность



МЕЧТАЮТ ЛИ АВТОБОТЫ ОБ АВТОТЕСТАХ?

Игорь Сахно

koronaitech